

Telematics Applications

Project – Development of a distributed application integrating Bicing, Telegram, IP Geolocation, Air Quality Index and Open Gateway services and deploying it in AWS

Bicing free slots /Air Quality notifier

Davinia Hernández-Leo, David Llanos Santos

Objectives

- Build a distributed application interacting with multiple real services and interfaces (Bicing¹, Telegram², AWS³, IP API⁴, Air Pollution Quality Index⁵, Open Gateway⁶).
- Communicate asynchronous processes.
- Work and learn real interfaces based on REST and JSON.
- Deploy services in the cloud (AWS).
- Build a small HTML page to make GET/POST requests using AJAX.

Description

The aim of this project is to implement a system that notifies users with two types of Telegram messages to user phones: one with the free slots of the Bicing stations the user has previously subscribed to and other with the air quality index (AQI) of the user location based on the IP. For this purpose, some external services will be used: Bicing, Telegram, AWS, IP API, Air Pollution Quality Index and Open Gateway.

The functionality of the application is as follows:

The main service is the **Bicing Free Slots/Air Quality Notifier (BAN)**. The requirements of this service are:

- **Access and save bicycles information from Bicing API.** This API must be accessed the first time a request is received and it must save the stations and their status during some configured time (cache). After expiring this time, the first request related to bicycles must recover the information from Bicing API again and save it to cache.
- **Allow users to subscribe to a list of Bicing stations using their phone number.** This service is **restricted to users aged 18 or older**, so it must use the **Verify Age API** from Open Gateway to validate the user's age before registering them in the system. If the provided `phone_number` belongs to a user who does not meet the age requirement, an **error must be returned**, and the user **must not be registered**.
- **Send a Telegram message to the users** with the free slots of the stations they have previously subscribed to.
- **Send a Telegram message to the users** with the Air Quality Index of the city where the IP the device making the request is connected.
- This service must **be deployed in an AWS EC2 instance** in the same way it was made in the last seminar.

To accomplish these requirements, the BAN must publish a **REST API** with three services producing and consuming messages in JSON format:

- **Stations service**, with one method:
 - **Get stations:** Returns the stations list with the same information retrieved from the Bicing API. A request to this method must return the cached information and not make a request to the Bicing service. If cache is empty, it must recover the information from Bicing API directly. That is, this method will return the same data during 120

¹ <https://www.bicing.cat/>

² <https://telegram.org/>

³ <https://aws.amazon.com/>

⁴ <https://ip-api.com/>

⁵ <http://aqicn.org>

⁶ <https://opengateway.telefonica.com/>

seconds, until cache expires. Then, when a new request to this service is made, if cache is empty or expired, a new request is made by the BAN to the Bicing API to update the cached information.

- **Clients service**, with two methods:

- **Subscribe client to stations:** This method is used to **create users** and **subscribe them to Bicing stations**. The service must receive:
 1. The **client's phone number**, in order to validate their age.
 2. The **list of station IDs** the client wishes to subscribe to.
 3. The **Telegram data** (the bot token and chat id) required to send a Telegram message.

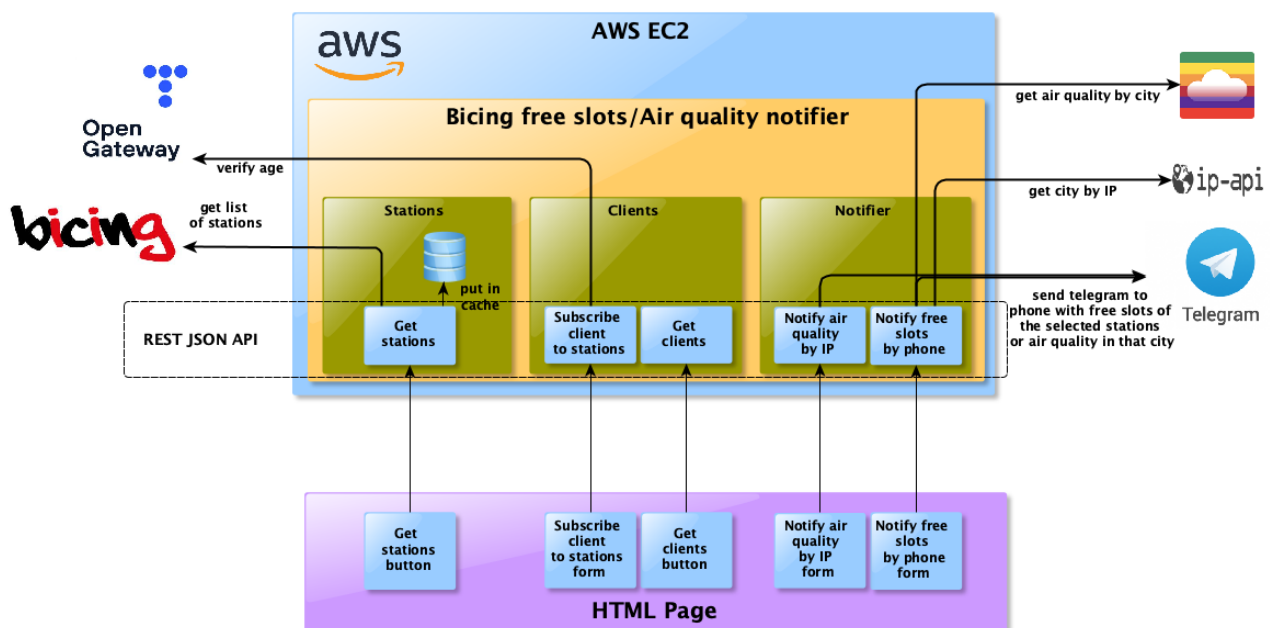
Before subscribing the client, the service must **verify the user's age** using the **Verify Age API** from Open Gateway.

- **Get clients:** Returns the list of clients subscribed to this service, including all the information of the users profile: the phone number, Telegram data (token and chat_id) of each user and the list of stations identifiers which they are subscribed to.

- **Notifier service**, with two methods:

- **Notify slots:** This method receives the phone number of the user and sends a telegram message with the free slots in each one of the stations the client previously subscribed to.
- **Notify Air Quality:** This method receives the IP of the client device and the phone number of the user and sends a telegram message with the real-time air quality information. It is recommended to “translate” the number to a conceptual level (<https://aqicn.org/scale/>).

The following figure shows a diagram of the operation of the application:



It is also a requirement to build a simple HTML page to show and validate the application functionality. This page should make GET/POST requests to all the methods and services offered by the BAN application and show the results in the HTML page. It is recommended to use AJAX requests to avoid reloading the whole page.

- Appendix sections should be followed to build the application.
- As a recommendation, the **BAN** service should be implemented service by service in this order: stations, clients and notifier.
- The Air Quality API is able to provide the Air Quality Index by IP, but it is a requirement to use a service to translate the IP to city name and other service to get Air Quality Index by city name.

Project assessment

The assesment criteria of the project are:

- **Intermediate demo (20% of the mark).** It will take place during the fourth project session: **20-2-2026**. Brief demo of the following methods of BAN API:
 - Get stations, subscribe clients to stations (no need to check the age) and get clients.
 - The API must be deployed in AWS.
 - You must use your own HTML page for the requests.
- **Final demo (50% of the mark).** It will take place during the final seminar session: **4-3-2026 for S101** and **6-3-2026 for S102**. It includes 2 parts:
 - Demonstration of the full functionality achieved in the application.
 - During the session, each group will be asked questions related to the assignment and required to apply minor changes to their implementation, which must be completed and demonstrated before the session concludes.
- **Structure, comments and quality of the source code (10% of the mark).** Fault management of the application.
- **Technical report (20% of the mark).** Maximum 4 pages. The report should include only these 2 sections:
 - Document the REST API of BAN: Method URL, parameters, examples of request and response with the JSON, etc.
 - Implementation: Description of how the most relevant parts of the project have been implemented, illustrated using a class diagram. The diagram should be created manually.

Project delivery: Each group will deliver a single zip file including the project with the source code and the technical report. This file will be uploaded to the activity created in Aula Global.

The file name will be the following: `at_project_NIAstudent1_NIAstudent2.zip`

The deadline to deliver the project outcome is 6th March 2026.

Marks behind the minimum required to pass the course will be published on 9th of March.

Appendix

Accessing Bicing API

Bicing is a service that spreads bicycles and bike stations all over the city for the inhabitants to use. It publishes an API with the status of all stations in <https://opendata-ajuntament.barcelona.cat/data/en/dataset/estat-estacions-bicing>, and another API with the stations information in <https://opendata-ajuntament.barcelona.cat/data/en/dataset/informacio-estacions-bicing>

This first service, available at <https://opendata-ajuntament.barcelona.cat/data/dataset/6aa3416d-cela-494d-861b-7bd07f069600/resource/1b215493-9e63-4a12-8980-2d7e0fa19f85/download>, returns the number of available bikes and docks. These data are enough for the purpose of the project but, if you want more information about the station (name, location with lat-long coordinates, street address and number), you could use the other service, whose API is published in <https://opendata-ajuntament.barcelona.cat/data/dataset/bd2462df-6e1e-4e37-8205-a4b8e7313b84/resource/f60e9291-5aaa-417d-9b91-612a9de800aa/download>

Starting from January 2024, accessing these services requires a token. To obtain access, registration on the website <https://opendata-ajuntament.barcelona.cat/> is mandatory. You will need to create a new account at the following link: <https://opendata-ajuntament.barcelona.cat/ca/user/register> Upon registration, you will receive a token which must be included in the authorization header to access these APIs (try to download a resource to get the token). Below is an example of how to access them using curl:

```
curl 'https://opendata-ajuntament.barcelona.cat/data/dataset/6aa3416d-cela-494d-861b-7bd07f069600/resource/1b215493-9e63-4a12-8980-2d7e0fa19f85/download' -H 'Authorization: YOUR_TOKEN'
```

Both services return a JSON object. You can parse the JSON returned by the services creating the same structure of the message using Java classes. It is not necessary to create all fields of the message, just the ones you want to use. For example, for the service publishing the available bikes and docks, the following classes structure could be created:

```
public class Data {
    private Stations data;

    public class Stations {
        private List<Station> stations;

        public class Station {
            private int station_id;
            private int num_bikes_available;
            private int num_docks_available;
            private long last_reported;
            private boolean is_charging_station;
            private String status;
```

Remember, as you did in the seminar, to add constructors (empty and with arguments), getters, setters and toString methods to the classes (use Source options from Eclipse or any other tool to build those methods).

Since a token is required, you must include this header in the request made from Java. Example:

```
Client client = ClientBuilder.newClient();
WebTarget targetGet = client.target(bicingURL).path(bicingPath);

Data data = targetGet.request(MediaType.APPLICATION_JSON_TYPE)
    .header("Authorization", token)
    .get(new GenericType<Data>() {});
```

Sending messages using Telegram bots

Telegram⁷ is a messaging app (cloud-based instant messaging service). Telegram clients exist for both mobile and desktop systems. Users can send messages and exchange photos, videos, stickers and files of any type.

Documentation:

- Telegram bots: <https://core.telegram.org/bots>
- Telegram bots api: <https://core.telegram.org/bots/api>

⁸ <https://telegram.org/>

To send a message using Telegram bots you must follow the next steps:

- Create a new bot using the BotFather.
 - Create a new chat with the BotFather, open this url with the Telegram app or the Telegram web client: <https://telegram.me/botfather>
 - Write `/newbot` in the chat and choose a name and a username for it.
 - Save the token generated for this bot.
 - Open a new chat clicking on the link of the BotFather chat or opening this url with the Telegram app or the Telegram web client: https://telegram.me/username_of_the_bot
- Once the chat with the new bot has been created, we need to get the id of the chat to be able to send messages to it using the API. To get the id of the chat, make a request to <https://api.telegram.org/botTOKEN/getUpdates> substituting TOKEN with your token and save the property: `result.message[0].chat.id` If the output is empty, you have to write at least one message in the chat.
- With the TOKEN and the chat ID, we will be able to send messages to the chat with the API. You can use this example:
 - Create the Message object we must send. As in the seminar, remember adding constructors, setters, getters and toString methods.

```
public class Message {  
    private long chat_id;  
    private String text;  
}
```

- Then, create a client to send an HTTP POST request with that object in the body:

```
Message message = new Message(chatID, "Message");  
WebTarget targetSendMessage = client.target("https://api.telegram.org").path("/botTOKEN/sendMessage");  
String response = targetSendMessage.request()  
    .post(Entity.entity(message, MediaType.APPLICATION_JSON_TYPE), String.class);
```

Relevant concepts of Telegram bots and chats:

- A bot does **NOT** receive messages “directly” via HTTP as if it were an endpoint. What you do over HTTP is **send messages from your bot to a chat** using the Telegram Bot API.
- In private chats, like the one we created, it is **NOT possible** to obtain the `chat_id` if the user has never sent a message to the bot. Telegram protects privacy in this way:
 - A bot cannot “discover” users.
 - The first message is always initiated by the user.
 - A bot only knows a user after that user sends a message.
- If you add the bot to a group chat, the bot receives the `my_chat_member` event, and then it is possible to access the `chat_id` of the group. Even without anyone sending a normal message, the “added” event already contains the `chat_id`.
- The `chat_id` identifies the chat, **not the bot**. And in private chats, `chat_id == user_id`, so if you create multiple bots, each will have a different token, but the `chat_id` will be the same.
 - If you send messages to two different chats created with bots from the same account, the `chat_id` will be the same because the chat is still the same (your user).
 - The token identifies the bot, and it is what Telegram uses to know **who is sending the message**.
 - Each bot has its own unique token.
 - Even if the `chat_id` is the same, Telegram knows which bot is sending the message thanks to the token you use.
 - An analogy would be: the bot is a **phone**, and the `chat_id` is **your phone number**. You call two different phones, but your number remains the same.

Accessing Open Gateway APIs

Open Gateway is an initiative spearheaded by the GSMA in collaboration with leading telecommunications operators, cloud providers, and aggregators worldwide. GSMA Open Gateway currently offers a variety of APIs, including those for **Identity and Fraud Prevention**, **Location Services**, and **Communication Services**: <https://opengateway.telefonica.com/>.

In our project, we will use only one of these APIs, **Verify Age Threshold**, to validate that the age of the subscriber associated with a phone number is **equal to or greater than a specified age threshold**:

<https://developers.opengateway.telefonica.com/reference/verifyage>

One of the objectives of this project is to gain hands-on experience with **real-world API development**. For this purpose, it is necessary to:

1. Read the **Open Gateway documentation**.
2. Sign up for an account and obtain the necessary **API credentials** to access the services in the Sandbox.

Your application must have access to the **Age Verification API**. In the usage mode, select **mock mode**. At the end of the setup process, you will have created an application with a **client ID** and **client secret**, which are required to obtain a token from the **AuthServer** to access the API.

As standardized by **CAMARA**, access to Open Gateway APIs is secured using **OpenID Connect (OIDC)** on top of the **OAuth 2.0** protocol. We will use the **backend authorization flow**, as we want to access Open Gateway APIs from our backend without end-user interaction, following the **OIDC CIBA (Client-Initiated Backchannel Authentication) flow**.

For this project, the API must be integrated via **HTTP requests directly**, i.e., **without using an SDK**.

Resources:

- Open Gateway Documentation: <https://developers.opengateway.telefonica.com/docs/initiative>
- Sandbox to create an application: <https://opengateway.telefonica.com/en/login> and <https://sandbox.opengateway.telefonica.com/>
- Register an application in the Sandbox: <https://developers.opengateway.telefonica.com/docs/usethesandbox>
- API integration: <https://developers.opengateway.telefonica.com/docs/apiintegration>
- Authorization: <https://developers.opengateway.telefonica.com/docs/authorization>
- Backend authorization flow: <https://developers.opengateway.telefonica.com/docs/backend>
- Verify Age API: <https://developers.opengateway.telefonica.com/reference/verifyage>

Getting city from IP

There are multiple services offering diverse information by IP. For the aim of this project, we are interested in getting the city name where the IP is located. For this purpose we will use the IP API service (<https://ip-api.com/>), but you are free to use any other service you prefer to get the city name by IP.

Read the IP API documentation to create a client able to recover the city name by the IP:

- API Overview: <https://ip-api.com/docs/api:json>

Getting air quality index from city name

There are also multiple services offering air quality information by city name, location coordinates, etc. For the aim of this project, we are interested in getting the air quality index (AQI) using the city name. For this purpose we will use the World Air Quality Index project: <http://aqicn.org/>, but you are free to use any other service you prefer to get the air quality index by city name. Once you register in the platform, you will get a token you will need to make requests to the API.

Read the Air Quality API documentation to create a client able to recover the Air Quality Index by the city name:

- API general information: <http://aqicn.org/api/>
- JSON API: <http://aqicn.org/json-api/doc/>

Making HTTP AJAX requests

You are free to build the HTML page to show and validate the application functionality with the technology/library/framework you prefer. In this section, some recommendations are given to build that functionality.

To make AJAX requests, you could follow the following steps:

- Create buttons to make the requests (also for the forms):

```
<button type="button" onclick="load(...)">Make request</button>
```

- Create the JavaScript function/s to be executed after clicking the button. This function will send the request and update the specified element of the html page. This code should be put between `<script>` `</script>` tags in the HTML page.

```
function load() {
    var xmlhttp = new XMLHttpRequest();
    xmlhttp.onreadystatechange = function() {
        if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {
            document.getElementById("sectionId").innerHTML = xmlhttp.responseText;
        }
    };
    xmlhttp.open("GET", "ban/stations/list", true);
    xmlhttp.send();
}
```

- In case you want to send a JSON object, you must get the values of the form elements (add the “id” attribute in the form elements), build a JavaScript dictionary and then, modify the request HTTP send method setting the content type header and transforming the dictionary to JSON in the sending method.

```
var params = {
    "phone": document.getElementById("phone").value,
    "telegramToken": document.getElementById("telegramToken").value,
    "stationsIds": document.getElementById("stationsIds").value.replace(/\s/g, "").split(",")
};
```

```
xmlhttp.setRequestHeader('Content-Type', 'application/json; charset=UTF-8');
xmlhttp.send(JSON.stringify(params));
```

- Create the html section you want the content to be updated with the tag div:

```
<div id="sectionId"></div>
```

Optional: Create log file to show application traces

If you are interested in creating a log file to show application traces, the easiest way is the following:

1 – Add log4j dependency to pom.xml:

```
<dependency>
    <groupId>log4j</groupId>
    <artifactId>log4j</artifactId>
    <version>1.2.17</version>
</dependency>
```

2 – Create a log4j.properties file in WEB-INF/classes folder:

```
# Root logger option
log4j.rootLogger=DEBUG, stdout, file

# Redirect log messages to console
log4j.appender.stdout=org.apache.log4j.ConsoleAppender
log4j.appender.stdout.Target=System.out
log4j.appender.stdout.layout=org.apache.log4j.PatternLayout
log4j.appender.stdout.layout.ConversionPattern=%d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n

# Redirect log messages to a log file, support file rolling.
log4j.appender.file=org.apache.log4j.RollingFileAppender
log4j.appender.file.File=/tmp/log4j-application.log
log4j.appender.file.MaxFileSize=5MB
log4j.appender.file.MaxBackupIndex=10
log4j.appender.file.layout=org.apache.log4j.PatternLayout
```

```
log4j.appender.file.layout.ConversionPattern=%d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n
```

3 – To create a trace, instance the Logger class with the CLASSNAME of your class:

```
final static Logger logger = Logger.getLogger(CLASSNAME.class);
```

And then add the trace where you want:

```
logger.debug("Users list: " + users);
```

4 – Traces are generated in /tmp/log4j-application.log file:

```
[ec2-user@ip-172-31-36-182 ~]$ tail -f /tmp/log4j-application.log
2019-02-16 13:37:18 DEBUG StationsService:43 - Request to get stations list
2019-02-16 13:37:18 DEBUG StationsService:44 - Updating biking stations at Sat Feb 16 13:37:18 UTC 2019
```